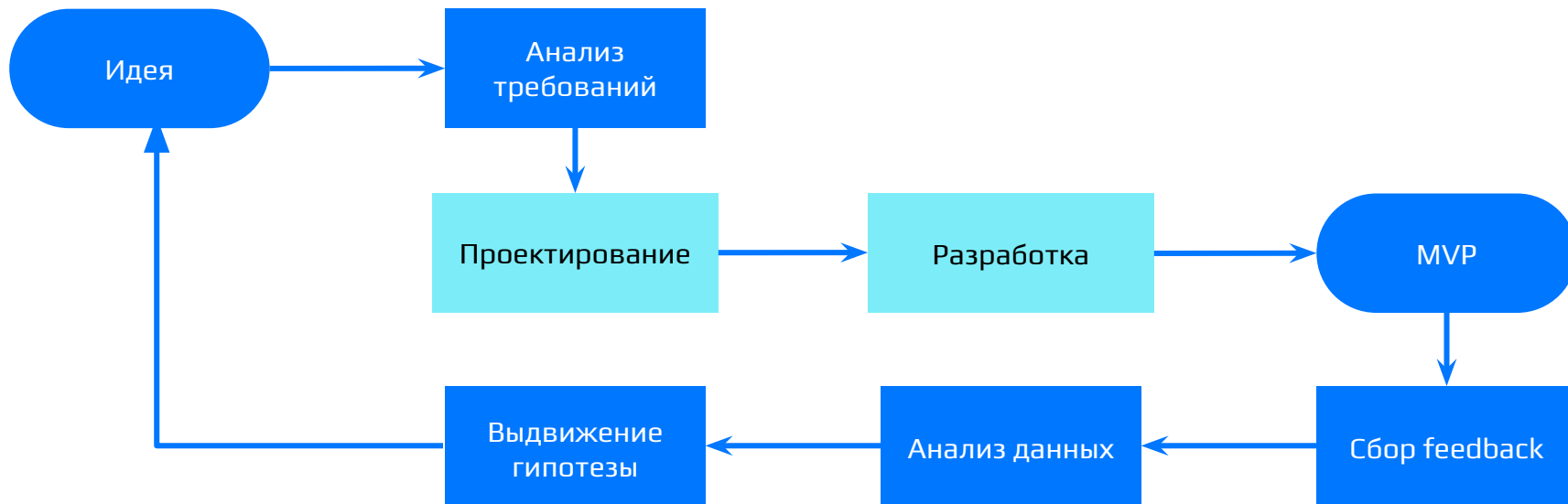


Подходы к разработке програмного обеспечения

Олег Гибадулин



От архитектуры к написанию кода



Роли

Управление продуктом и процессами

- **Product Manager (PM):** Отвечает за бизнес-успех. Решает, ЧТО и ЗАЧЕМ мы делаем (метрики, рынок, фичи).
- **Project Manager** (Руководитель проектов): Отвечает за доставку. Следит за сроками, бюджетом и ресурсами (чтобы всё было сделано вовремя).
- **Scrum Master:** Настройщик процессов. Фасилитирует встречи, снимает блокеры, учит команду работать по Agile.
- **Engineering Manager:** Руководитель руководителей (тимлидов). Управляет отделами разработки, наймом и бюджетами.

Аналитика и Дизайн

- **Business Analyst** (BA): Исследует бизнес-процессы компании или заказчика, ищет точки роста и оптимизации.
- **System Analyst** (SA): Переводит "хотелки" бизнеса в строгие технические требования (ТЗ, API-контракты) для программистов.
- **UX/UI Designer**: Проектирует логику пользовательского пути (UX) и отрисовывает визуальный интерфейс (UI).
- **UX Researcher**: Общается с пользователями, чтобы доказать, что дизайн удобен.
- **Data Analyst**: Достает из базы данных цифры (как пользователи ведут себя в приложении) и превращает их в понятные бизнесу графики.

Разработка

- **Team Lead:** Лидер команды. Отвечает за людей, их рост, мотивацию и микроклимат.
- **Tech Lead:** Технический лидер. Принимает финальные решения по архитектуре, технологиям и безопасности.
- **Frontend/Mobile Developer:** Пишет клиентскую часть приложения.
- **Backend Developer:** Пишет серверную логику, алгоритмы и выстраивает работу с бд.
- **Fullstack Developer:** Человек-оркестр, способный писать и Frontend, и Backend.
- **Data Scientist / ML Engineer:** Обучает нейросети, пишет алгоритмы рекомендаций.
- **Data Engineer:** строит ETL-пайплайны для формированию данных нужного вида.

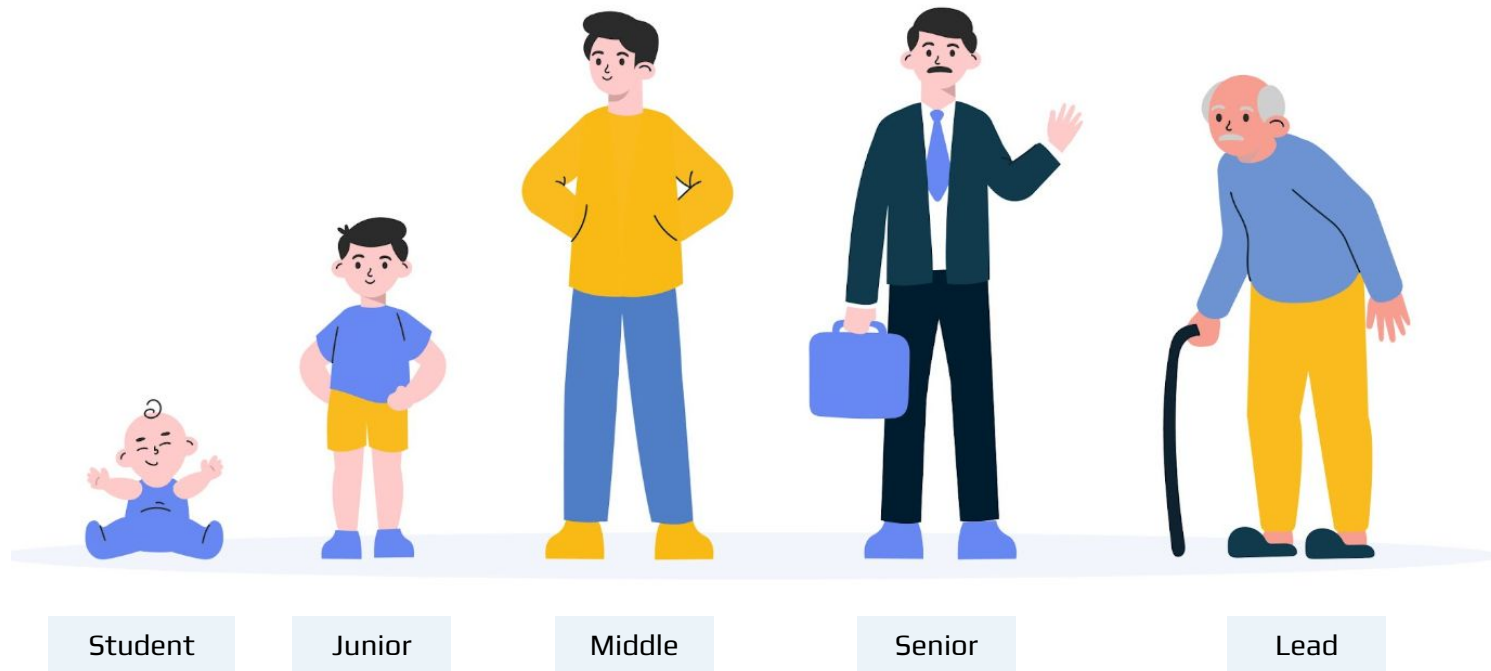
Качество

- **QA Manual** (Ручной тестировщик): Проходит пользовательские сценарии руками, ищет баги и ломает логику.
- **QA Automation** (AQA): Программист, который пишет код, автоматически тестирующий другой код (автотесты).

Инфраструктура

- **DevOps Engineer:** Автоматизирует рутину. Настраивает пайплайны (CI/CD), чтобы код разработчика сам собирался и доставлялся на сервер.
- **SRE** (Site Reliability Engineer): Отвечает за то, чтобы сервера выдерживали огромную нагрузку и продукт не "падал".
- **DBA** (Database Administrator): Проектирует и оптимизирует базы данных, чтобы поиск информации занимал миллисекунды.
- **Security Engineer** (AppSec/DevSecOps): Белый хакер. Защищает продукт от взломов, утечек данных и DDOS-атак.

Карьерная карта



Какие навыки у разработчиков?

Ключевые навыки программиста

- Хард скилы
- Софт скилы

Ключевые навыки программиста

- Технические навыки
 - Знание инструментов (ЯП, инфраструктура)
 - Знание ландшафта (экосистема индустрии, ЯП)
 - Знание предметной области
 - Навык манипуляции данными
 - Навык “как это работает” в технике
 - **Бизнес-ориентированность :)**

Ключевые навыки программиста

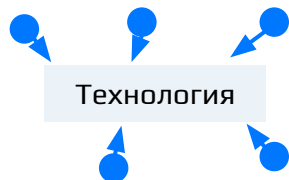
- Социальные навыки
 - Навык “как это работает” в социуме / проекте / продукте
 - Личная эффективность
 - Организованность
 - Управление рисками
 - Коммуникации
 - Правильно выражать свои мысли
 - Общение
 - Дипломатия
 - **Бизнес-ориентированность :)**

Типология команд

Типология команд

1

Функциональные
команды



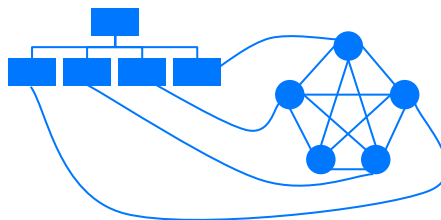
2

Проектные
(матричные)
команды



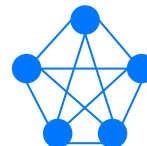
3

Кросс-функциональные
команды



4

Самоорганизующиеся
команды



Функциональные команды

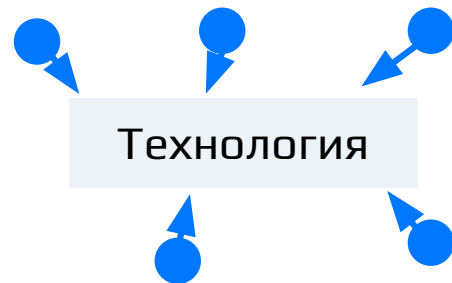
Это группировка по технологическому стеку. Отлично работает для Platform / Component Teams (например, «Команда платформенной инфраструктуры», «Команда дизайн-системы»).

Плюсы:

- Глубокая экспертиза и топовые стандарты кода

Минусы:

- Жесткая бюрократия на стыках команд
- Технологии ради технологий

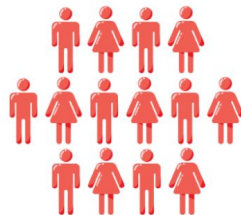


Кросс-функциональные команды

Функциональная команда



Системные аналитики



Тестировщики



Разработчики

Кросс-функциональная команда



Проектная команда

Кросс-функциональные команды

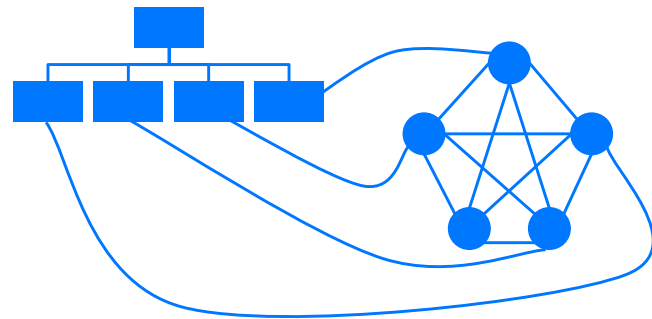
Фокус: На пользовательской ценности (например: команда "Корзина", команда "Лента новостей").

Плюсы:

- Низкий Time-to-Market.
- Фокус на бизнесе, а не на технологиях ради технологий.

Минусы:

- Риск "изобретения велосипедов" (каждая команда может написать свою кнопку или свой сетевой слой).



Матричные (и проектные) команды

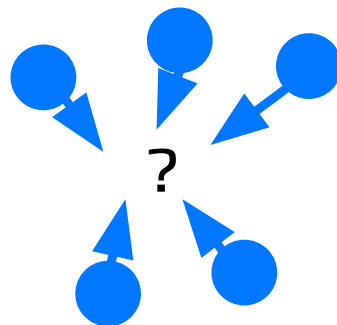
У инженера возникает двойное подчинение: функ и кросфунк лид.

Плюсы:

- Идеальный баланс
- Понятный вектор роста

Минусы:

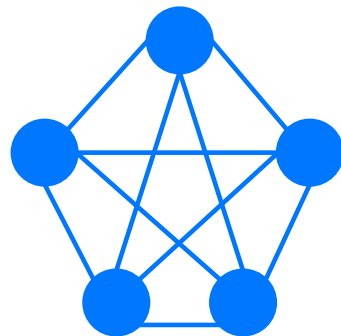
- Конфликт интересов
- Коммуникационный оверхед



Самоорганизующиеся команды

Это не вид оргструктуры, а **высшая ступень эволюции любой Agile-команды**. Это состояние, когда команде больше не нужен микроменеджмент. Лидерство внутри динамически переходит от человека к человеку в зависимости от задачи.

Бизнес спускает команде цель («Нам нужно увеличить конверсию в покупку на 5%»), а как именно это сделать и кто что будет писать — команда решает сама.



Закон Конвея

“Организации проектируют системы, которые копируют структуру коммуникаций в этой организации”

Вывод: Нельзя поменять архитектуру приложения, не поменяв структуру общения в команде.

Процессы разработки

- Традиционные
- Гибкие

Почему нужны разные процессы

Адаптация к потребностям проекта:

- Уровни сложности
- Размер проекта
- Временные рамки
- Бюджетные ограничения

Адаптация к потребностям команды:

- Распределенные команды против офисных команд
- Уровни опыта
- Стили общения
- Предпочтения в области совместной работы

Waterfall - последовательный подход

1

Определение
требований

2

Проектирование

3

Реализация

4

Тестирование

5

Внедрение

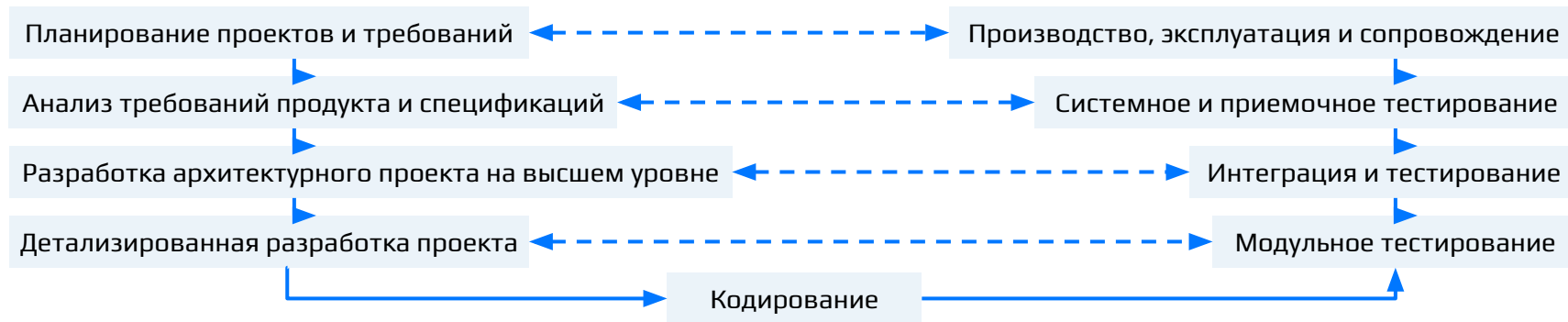
Плюсы:

- Простота и легкодоступность для понимания
- Хорошо документированные шаги
- Четкие результаты и контрольные точки

Минусы:

- Жесткая структура, трудно адаптируемая к изменениям
- Проблемы обнаруживаются поздно
- Работающее программное обеспечение отсутствует до конца процесса

V-модель



Плюсы:

- Поощряет детальное планирование, снижая риски проекта
- Простой и понятный, что делает его доступным для широкого круга проектов

Минусы:

- Жесткая структура, трудно адаптируемая к изменениям
- Высокие первоначальные требования: Требуется тщательная документация с самого начала

Итерационные процессы

1

Разбиение всего проекта на небольшие интервалы (итерации)

2

В конце каждой итерации функционирующий промежуточный результат

3

Возможность получения обратной связи после каждой итерации

4

Эволюционное развитие: каждая новая итерация строится на базе предыдущей

Agile

Манифест Agile:

- Индивидуумы и взаимодействие выше процессов и инструментов
- Работающий продукт выше всесторонней документации
- Сотрудничество заказчика и команды выше условий контракта
- Готовность к изменениям выше следования плану

Основные принципы Agile:

- Удовлетворенность клиентов
- Приветствуем изменение требований
- Поставляйте работающее программное обеспечение
- Тесное, ежедневное сотрудничество
- Проекты строятся вокруг мотивированных людей
- Беседа лицом к лицу
- Гибкие процессы способствуют устойчивому развитию
- Постоянное внимание к техническому совершенству
- Регулярно команда размышляет о том, как стать более эффективной

SCRUM

- SCRUM - это фреймворк гибкой разработки, используемый для управления и улучшения работы в командных проектах.
- Он подчеркивает практические результаты и гибкость в адаптации к изменениям.
- Подходит для сложных проектов, где требования меняются и риски необходимо свести к минимуму.
- Способствует итеративному прогрессу посредством коротких рабочих циклов, известных как спринты

SCRUM

Роли:

- Владелец продукта
- Scrum-мастер
- Команда разработчиков

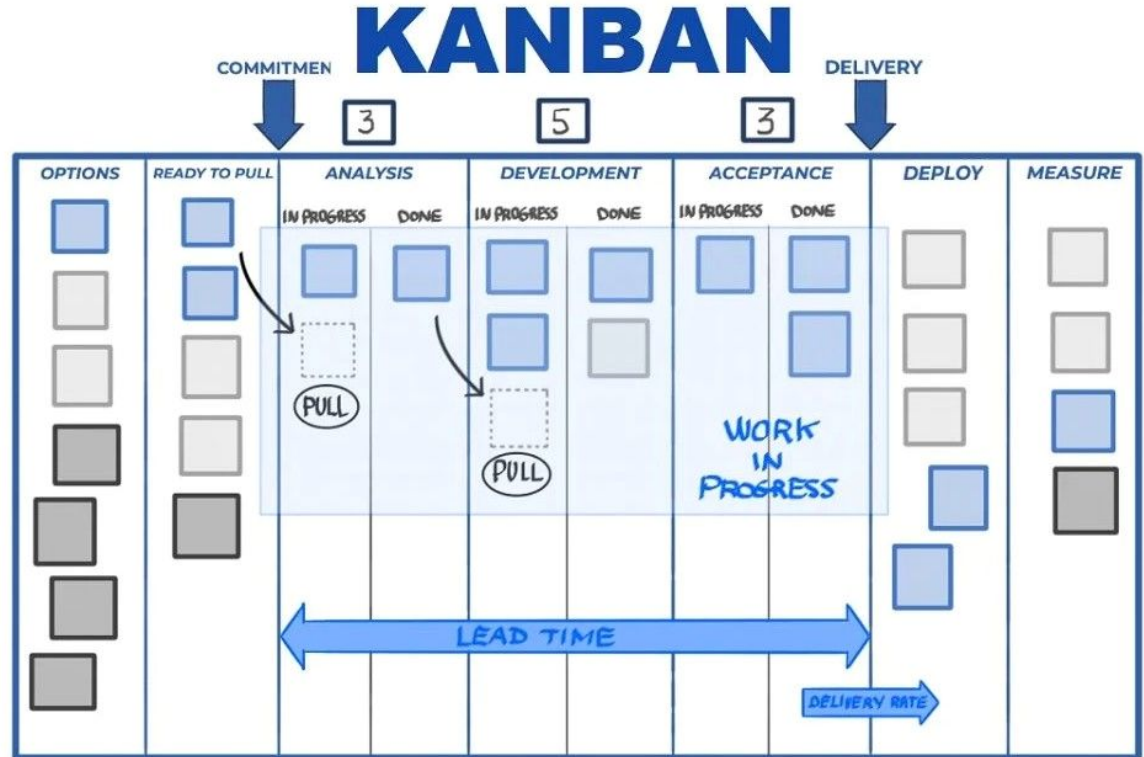
Мероприятия:

- Планирование спринта
- Ежедневный Scrum
- Обзор спринта
- Ретроспектива спринта

Артефакты:

- Список невыполненных работ по продукту
- Список невыполненных задач в спринте
- Прирост задач

Kanban



Kanban

Ключевые концепции:

- Визуализируйте рабочий процесс
- Ограничьте количество незавершенных работ (WIP)
- Управление потоками
- Четко сформулируйте политику процессов
- Циклы обратной связи
- Совместное совершенствование

Визуализация работы:

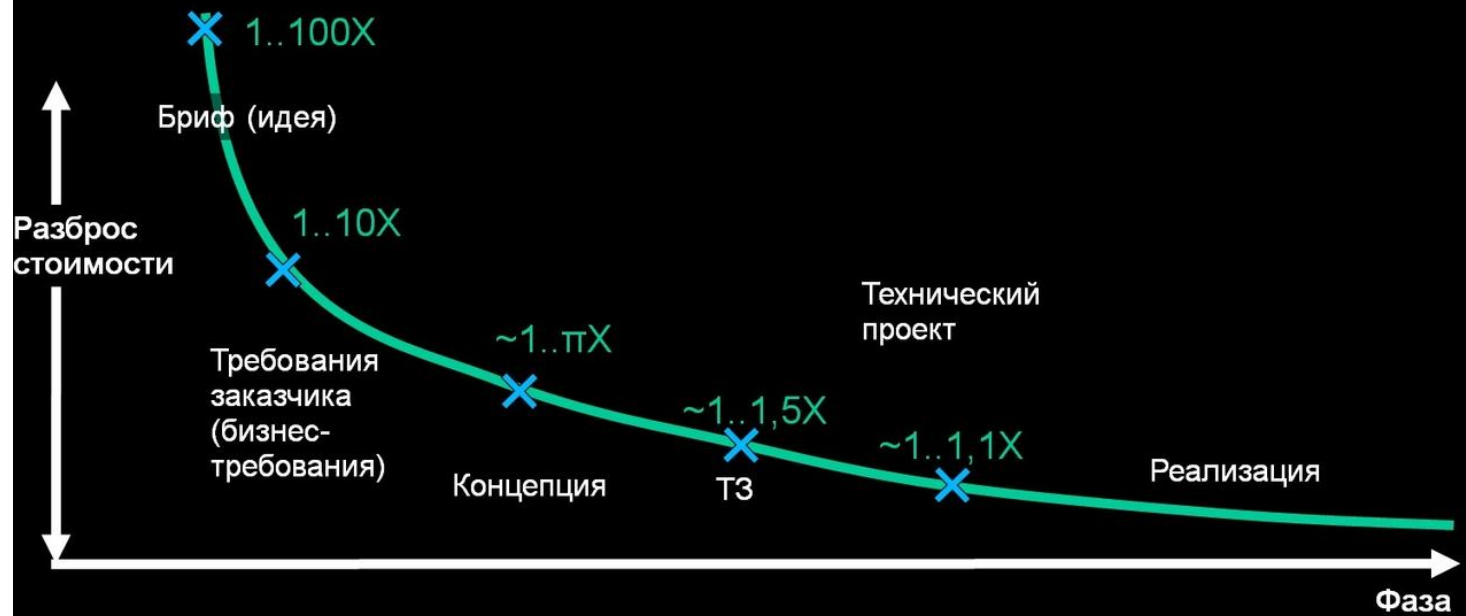
- Доска Канбан: Инструмент, используемый для визуализации работы и рабочего процесса.
- Столбцы: Представляют этапы рабочего процесса (например, "Сделать", "В процессе", "Готово").
- Карточки: Представляют задачи или рабочие элементы.
- Пределы WIP: Указаны для каждого этапа, чтобы предотвратить перегрузку этапов работы.

Главное правило доски

- **Узкие горлышки** (Bottlenecks): Доска нужна не для контроля, а для поиска проблем. Если в Code Review висит 10 задач, а в In Progress ноль — команда простаивает. Разработчикам нужно бросить писать новый код и пойти проверять чужой.
- **Переходы**: Задача не может прыгнуть из To Do сразу в Done. Она обязана пройти все этапы контроля качества.

Оценка

Конус неопределенности



Почему мы не оцениваем задачи в часах?

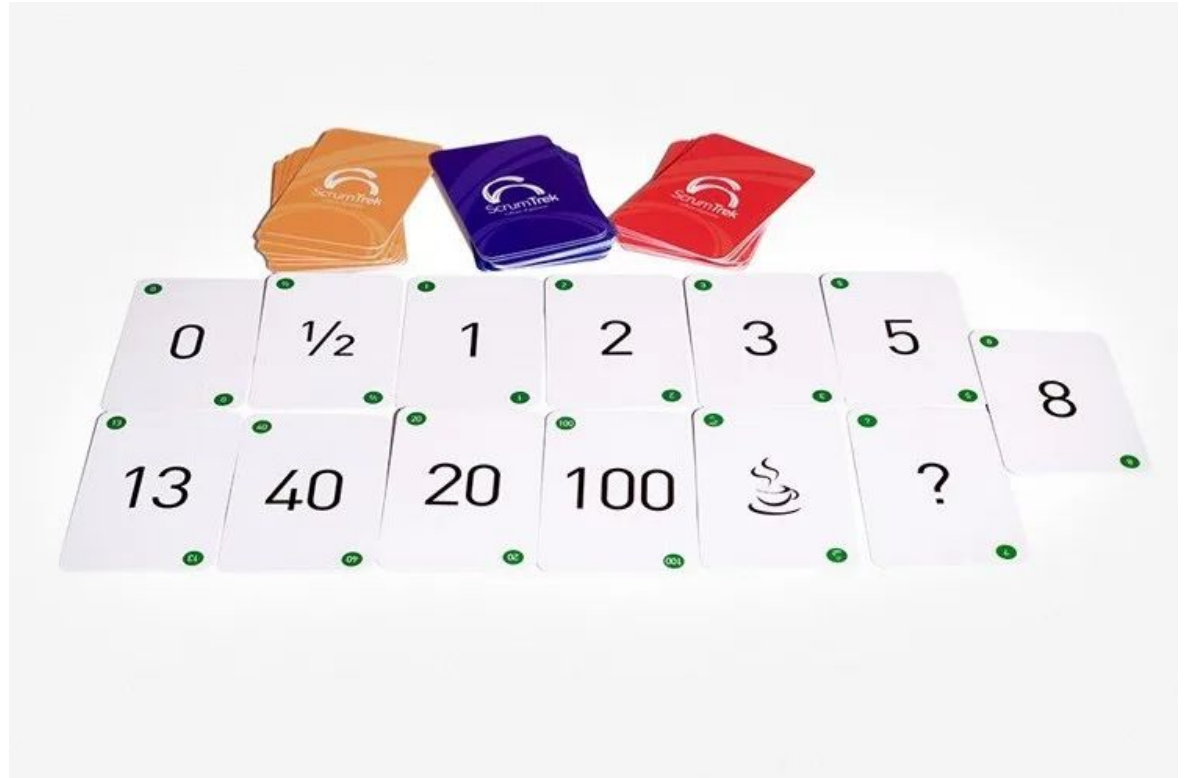
Проблема: Оценка в часах заставляет людей врать или перерабатывать.

Story Points

Абстрактная величина. Учитывает три фактора:

- **Объем работы:** Покрасить 1 кнопку (1 SP) vs. покрасить 100 кнопок (5 SP).
- **Сложность:** Вывести текст на экран (1 SP) vs. написать сложный математический алгоритм (5 SP).
- **Риск / Неизвестность:** Если мы никогда раньше не интегрировали эту библиотеку и не знаем, как она себя поведет — накидываем баллы за риск (8 SP).

Planning Poker



Взаимодействие и коммуникации

Взаимодействие и коммуникации

1

Сотрудничество между
ролями – ключевой
фактор успеха

2

Постоянное взаимодействие
с заказчиком – залог успешной
разработки

3

Обратная связь помогает быстро
реагировать на изменения

4

Ведение документации помогает
фиксировать требования и решения

Приложения для коммуникации

1

Совместная
работа



2

Планирование



3

Дизайн



4

Разработка



5

Тестирование



6

Деплой



7

Мониторинг



Практика

Практическое задание №5

- **Hiring Plan:** Соберите виртуальную команду под ваш MVP. Напишите 1-2 главные зоны ответственности для каждой роли.
- **Development Framework:** Выберите методологию доставки. Напишите 2-3 тезиса: почему именно она подходит вашему продукту.
- **Team Rituals:** Спроектируйте обязательные встречи. Укажите: цель и частоту.

Спасибо за внимание!

Не забудьте оставить отзыв :)

